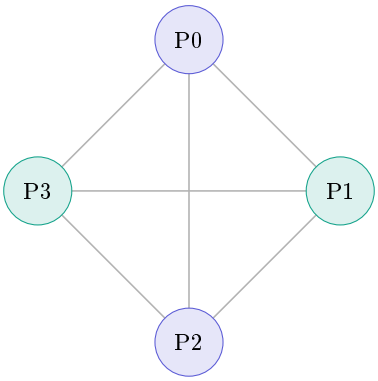
webSPDZ & Surplus

Secure Multi-Party Computation in the Browser — a Study Guide and Project History



A technical deep-dive into a browser-based MPC engine:
the ATLAS protocol, the compilation pipeline, the Docker deployment,
and the complete evolution of this fork from first clone to demo.

Prepared as presentation and self-study material.
Repository: `webSPDZ` (fork of MP-SPDZ via tbuchs/webSPDZ).
Compiled October 23, 2026.

Contents

1 Introduction **3**
- 1.1 Why this document exists . . . 3
- 1.2 What ships in this repository, in one paragraph . . . 3

2 Secure Multi-Party Computation **4**
- 2.1 The problem MPC solves . . . 4
- 2.2 Shamir secret sharing . . . 4
- 2.3 Security models: who can be corrupted, and how . . . 4

3 The ATLAS Protocol **6**
- 3.1 Why ATLAS . . . 6
- 3.2 The double-sharing trick . . . 6
- 3.3 Reading it in the actual source . . . 6

4 From MP-SPDZ to webSPDZ **8**
- 4.1 MP-SPDZ in one page . . . 8
- 4.2 The core idea of webSPDZ . . . 8
- 4.3 Compiling C++ to WebAssembly: the flags that matter . . . 8
- 4.4 Keeping the UI thread responsive . . . 9
- 4.5 Networking: WebRTC data channels and the signaling server . . . 9
- 4.6 Porting the supporting libraries . . . 10

5 The Compilation Pipeline **11**
- 5.1 Stage 1 — the program compiler: `.mpc` to bytecode . . . 11
- 5.2 Stage 2 — the protocol VM: C++ to native or to WASM . . . 12
- 5.3 No local toolchain needed to just *run* the demo . . . 12

6 The Surplus Matching Application **14**
- 6.1 The business problem . . . 14
- 6.2 Why this needs to be "data-oblivious" . . . 14
- 6.3 Worked example . . . 16

7 System Architecture of the Demo **17**
- 7.1 Three components, one trust boundary . . . 17
- 7.2 The argv/input injection mechanism . . . 17
- 7.3 The lobby protocol . . . 18

8 Docker Deployment **20**
- 8.1 Why the image can be *serve-only* . . . 20
- 8.2 Two services, one image . . . 20
- 8.3 Running it . . . 21

9 Project History **22**
9.1 Lineage . . . 22
9.2 Commit 1 — `ba257011`: the application is born . . . 22
9.3 Commit 2 — `56734a97`: housekeeping . . . 23
9.4 Commit 3 — `087c399a`: Dockerization . . . 23
9.5 Commit 4 — `3bf9c837`: the input-at-runtime prototype . . . 23
9.6 Commit 5 — `f99b28d6`: the lobby . . . 23
9.7 Commit 6 — `b160b884`: the beginner's guide . . . 23
9.8 Commit 7 — `d593fadd`: the lobby redesign . . . 23
9.9 Commit 8 — `cfabfe67`: the explainer page . . . 24
9.10 What this fork changed vs. what it inherited, in one table . . . 24

10 Security Discussion and Limitations **25**
10.1 What is actually revealed . . . 25
10.2 Known limitations . . . 25

11 Study Guide and Q&A **26**
11.1 One-paragraph answers, ready to say out loud . . . 26
11.2 Glossary . . . 27

A Command Reference **28**

B References **29**

CHAPTER 1

Introduction

1.1 Why this document exists

This guide was written to prepare a presentation on a repository (`webSPDZ`) that is three layers of history deep: it is a fork of a fork. Understanding it requires three separate bodies of knowledge that rarely show up in the same place:

1. **The cryptography**: what secure multi-party computation (MPC) is, and specifically how the **ATLAS** protocol shares and recombines secret values.
2. **The systems engineering**: how a C++ virtual machine gets compiled to WebAssembly with Emscripten, how two browser tabs talk to each other without a trusted server, and how a two-stage compiler (Python → bytecode → native/WASM machine) fits together.
3. **The project's own history**: what this specific fork changed on top of upstream `webSPDZ`, in what order, and why — i.e. the difference between "MP-SPDZ", "webSPDZ", and the **Surplus** application built on top of it in this repository.

Each of these is covered in its own chapter, and Chapter 9 reconstructs the project's evolution directly from the git history so that the presentation can show a believable before/after story.

1.2 What ships in this repository, in one paragraph

webSPDZ takes **MP-SPDZ** [2] — a native, general-purpose MPC framework written in C++ with a Python compiler front-end — and recompiles its protocol virtual machines to WebAssembly, so each computing party is an ordinary browser tab instead of a process on a server. On top of that engine, this fork builds **Surplus**: a four-party demo in which each party (framed as a "country") enters four private resource levels, and the four browsers jointly compute — without revealing those levels to anyone — which pairing of countries best complements each other's shortages, telling each party only the identity of its own best partner. The underlying MPC protocol is **ATLAS** [3], an honest-majority, Shamir-secret-sharing-based protocol chosen because it matches the four-party, one-corruption threat model of the demo.

CHAPTER 2

Secure Multi-Party Computation: the Theory

2.1 The problem MPC solves

Four parties each hold a private input. They want to compute a public function of all four inputs (or, as here, a *different* private output per party) without any party learning anything about another party's input beyond what the output necessarily implies. No party is trusted with the others' raw data, and in the protocols used here, no *external server* is trusted either — the two auxiliary servers in this project (Section 7) only route network traffic; they never see a plaintext value or a usable share of one.

2.2 Shamir secret sharing

The protocol used in this project (ATLAS) is built on **Shamir secret sharing**. To share a secret value x among n parties so that any $t+1$ of them can reconstruct it but any t learn nothing, the dealer picks a random polynomial of degree t

$$f(z) = x + a_1 z + a_2 z^2 + \cdots + a_t z^t, \qquad a_1, \ldots, a_t \xleftarrow{\$} \mathbb{F},$$

and gives party i the evaluation $f(i)$. Because $f(0) = x$, reconstructing x from any $t+1$ points is Lagrange interpolation. Two properties make this useful for computation, not just storage:

- **Addition is free.** If f shares x and g shares y, then $f + g$ shares $x + y$, and each party can add its two local evaluations without talking to anyone.
- **Multiplication raises the degree.** The pointwise product $f \cdot g$ is a polynomial of degree $2t$ sharing xy — correct, but shared at the wrong degree to keep composing. Every practical Shamir-based MPC protocol exists mainly to solve this *degree-reduction* problem efficiently.

2.3 Security models: who can be corrupted, and how

Protocols are classified along two independent axes:

Corruption threshold. Honest majority ($t < n/2$) vs. **dishonest majority** (up to $n-1$ parties corrupt). webSPDZ ships protocols for both (Table 2.1); this project's demo uses an honest-majority protocol with $n = 4,\ t = 1$.

Corruption behaviour. Passive/semi-honest corrupted parties follow the protocol exactly but try to learn extra information from what they see. **Active/malicious** corrupted parties

may send arbitrary, incorrect messages. ATLAS, as used here, is a *passive-security* protocol: it protects against parties that are nosy, not against parties that actively lie.

Table 2.1: MPC protocols shipped in this webSPDZ fork (from `README.md`).

Threshold	Protocol	Corruption model
Honest majority	Replicated-Ring	Passive
Honest majority	Rep4-Ring	Active
Honest majority	Shamir	Passive
Honest majority	Malicious Shamir	Active
Honest majority	**ATLAS**	**Passive (used by this demo)**
Dishonest majority	Semi2k	Passive

CHAPTER 3

The ATLAS Protocol

3.1 Why ATLAS

ATLAS [3] is an honest-majority protocol designed around a single idea: make the Shamir multiplication/degree-reduction step as cheap as possible by precomputing random *double sharings* and letting one designated party ("the king" for that multiplication) do the degree reduction, instead of running a full interactive resharing sub-protocol for every multiplication. This project uses it with $n = 4$ parties and threshold $t = 1$: the computation stays private as long as at most one of the four browser tabs is controlled by an adversary.

3.2 The double-sharing trick

A *double sharing* is a pair (r, r') of independently random values where r is Shamir-shared at degree t *and* r' is a sharing of the *same* value at degree $2t$. Given one of these, reducing a degree-$2t$ product $z = xy$ back to degree t needs only:

1. Every party locally computes its share of $z + r'$ (still degree $2t$) and sends it to the king for this multiplication.
2. The king, who alone can interpolate the degree-$2t$ polynomial from $2t+1$ shares, reconstructs the *masked* value $z + r'$ in the clear, then re-shares it at degree t to everyone.
3. Every party subtracts its share of r (the degree-t half of the same double sharing) from what the king sent, obtaining a degree-t sharing of z.

Because r is uniformly random and never reused, the king never learns z itself — only the one-time-padded value $z + r'$. The double sharings themselves are produced from random Shamir sharings contributed by every party, not by the king alone, so no single party chooses r. The king role is round-robined across parties (see `next_king` below) so that trust in any one "king" per multiplication is spread over the whole computation.

3.3 Reading it in the actual source

This is not merely textbook description — it is exactly what `Protocols/Atlas.hpp` in this repository implements. The class comment states it directly:

```
/**
 * ATLAS protocol (simple version).
 * Uses double sharings to reduce degree of Shamir secret sharing.
 */
template<class T>
class Atlas : public ProtocolBase<T>
```

```
{
    Shamir<T> shamir, shamir2;     // degree-t and degree-2t Shamir instances
    vector<array<T, 2>> double_sharings;
    int next_king, base_king;
    ShamirInput<T> resharing;
    ...
```

Listing 3.1: `Protocols/Atlas.h`: the one-line protocol summary

The multiplication pipeline follows the classic MP-SPDZ *prepare / exchange / finalize* pattern used by every protocol in the framework, so that the same program bytecode can run unmodified on ATLAS, Shamir, Semi2k, or any other protocol:

```
template<class T>
void Atlas<T>::prepare_mul(const T& x, const T& y, int)
{
    prepare(x.local_mul(y));       // local degree-2t product share
}

template<class T>
void Atlas<T>::prepare(const typename T::open_type& product)
{
    auto r = get_double_sharing();        // (r at degree t, r' at degree 2t)
    (product + r[0]).pack(oss2[next_king]); // send masked product to this round's
        king
    next_king = (next_king + 1) % P.num_players();
}

template<class T>
T Atlas<T>::finalize(...)
{
    T res = resharing.finalize(base_king) - masks.next(); // subtract r, degree t
    base_king = (base_king + 1) % P.num_players();
    return res;
}
```

Listing 3.2: `Protocols/Atlas.hpp`: multiplication via one double sharing per product (simplified)

Where this lives in the build

`atlas-party.x` (and its web build, `atlas-party.wasm`) links against `GC/AtlasSecret.o` and the templated `Protocols/Atlas*.h{,pp}` headers, exactly like every other `*-party.x` target links against its own protocol's headers (see the `Makefile` fragment in §5.2). The protocol code itself is untouched by webSPDZ or by this fork — it is inherited unchanged from MP-SPDZ. What this project changes is everything *around* it: how the VM is compiled, how parties find each other, and how input reaches it. That distinction — unmodified cryptographic core, heavily modified delivery — is the single most important fact to lead with in a presentation.

CHAPTER 4

From MP-SPDZ to webSPDZ: Porting a Native MPC Engine to the Browser

4.1 MP-SPDZ in one page

MP-SPDZ [2] is a general-purpose MPC framework: a Python-embedded domain-specific language compiles programs operating on secret types (`sint`, `sfix`, ...) down to a bytecode, and a family of over 40 protocol-specific C++ virtual machines (`mascot-party.x`, `shamir-party.x`, `semi2k-party.x`, ...) execute that same bytecode under different security models. Every party runs one native OS process, connected to the others over plain TCP sockets. This repository's `README_MPSPDZ.md` is the upstream documentation for that native system; `README.md` documents only what webSPDZ changes on top of it.

4.2 The core idea of webSPDZ

webSPDZ [1] replaces two things in that picture, and nothing else:

1. The native binary becomes 64-bit **WebAssembly**, compiled with **Emscripten** instead of a system C++ compiler.
2. Native TCP sockets become **WebRTC data channels** (peer-to-peer, browser to browser), set up with help from a small **signaling server**; WebSockets are offered as a less-private fallback.

Everything upstream of the network layer — the Python compiler, the bytecode format, the protocol logic in `Protocols/`, the comparison/arithmetic primitives in `Compiler/` — is the same code MP-SPDZ already had.

4.3 Compiling C++ to WebAssembly: the flags that matter

Building for the web is controlled by a `WEB=1` branch in the `Makefile` (see §5.2). The important Emscripten flags, and why each one is needed, are:

Emscripten is pinned to **4.0.6** across this project. Later versions were found to break either the WebRTC JS bindings or the linking of the prebuilt `local/wasm64/lib/*.a` libraries. This is recorded in `CONFIG.mine` and in the pre-session PDF `webSPDZ-ATLAS-changes.pdf`; do not casually `emsdk update` this repository.

Table 4.1: Key Emscripten/webSPDZ compiler and linker flags.

Flag	Purpose
`-sMEMORY64=1`	Generates 64-bit WASM (`wasm64`) code, needed for MP-SPDZ's memory footprint.
`-sWASMFS`	Uses Emscripten's newer, more modular virtual filesystem, used both for the `-embed-file` inputs and for the runtime `FS` writes described in §7.2.
`-sPROXY_TO_PTHREAD`	Moves the whole application (not just a worker) onto a dedicated application thread, so blocking network waits never freeze the page's UI thread.
`-sUSE_PTHREADS -sPTHREAD_POOL_SIZE=20`	Pre-spawns a pool of worker threads so that creating a new (p)thread inside the application thread does not have to wait on the UI thread each time.
`-fwasm-exceptions`	Uses native WASM exception handling instead of Emscripten's JS-based emulation; required because Shamir's setup code relies on catching a C++ exception, which is otherwise fatal under WASM.
`-sEXPORTED_RUNTIME_METHODS=FS,callMain`	Exposes Emscripten's filesystem and `main()`-invocation API to page JavaScript — the hook this fork uses for runtime input injection (§7.2).
`-embed-file`, `-shell-file`	Bake `Player-Data` / `Programs` into the WASM bundle at link time, and generate the page from a custom HTML template instead of Emscripten's default shell.
`-sASSERTIONS`, `-sEXCEPTION_CATCHING_ALLOWED` (debug only)	Emulate otherwise-unsupported WASM features for better error messages; left off in release builds because they slow the runtime down.

4.4 Keeping the UI thread responsive

A browser tab has one UI thread. MP-SPDZ's own code blocks synchronously while waiting for network messages from other parties — exactly the kind of call that would freeze a tab's UI for the whole computation. webSPDZ avoids this with `-sPROXY_TO_PTHREAD`: the UI thread's only job becomes spawning one application thread, inside of which the C++ program runs (and may itself spawn further worker threads from the pre-built pool). The one place this breaks down is the WebRTC API itself: several of its JavaScript objects are only reachable from the *Window* context, i.e. the UI thread, so Emscripten's `proxying.h` is used to hop back to the UI thread for just those calls, at some performance cost.

4.5 Networking: WebRTC data channels and the signaling server

Once compiled, parties cannot simply `connect()` to a socket — browsers have no raw TCP access. webSPDZ instead uses **WebRTC** data channels (`deps/datachannel-wasm`) for true peer-to-peer transport. Establishing a WebRTC channel, however, needs an out-of-band *signaling* step to exchange connection offers/answers/ICE candidates before the direct channel exists; that step is handled by `signaling_wss_server.js`, a small Node.js secure WebSocket server. Crucially:

- The signaling server only ever relays opaque WebRTC handshake messages (`login` / `offer` / `answer` / `candidate`, keyed by `group`+`name`). It never sees protocol traffic.
- Once the peer connections are open, all MPC traffic flows directly browser-to-browser; the signaling server could disconnect at that point without affecting the running computation.

- As a fallback (e.g. where WebRTC is unavailable), `websocket_server.js` offers plain WebSocket relaying instead — functionally correct, but no longer peer-to-peer, so it is explicitly documented as less secure.

4.6 Porting the supporting libraries

MP-SPDZ depends on **libsodium** and **OpenSSL**. Both needed a WASM build: libsodium ships an Emscripten build script (`./dist-build/emscripten.sh -sumo`, plus `-sMEMORY64=1`); OpenSSL needed a manual `emconfigure` / `emmake` pass with hand-fixed compiler paths. The resulting `.a` archives are committed under `local/wasm64/lib/`, which is precisely why a fresh clone can serve a working demo without ever running Emscripten (§8).

CHAPTER 5

The Compilation Pipeline

Understanding "how compiling works" in this project means understanding *two independent compilers* that run one after the other, for entirely different reasons.

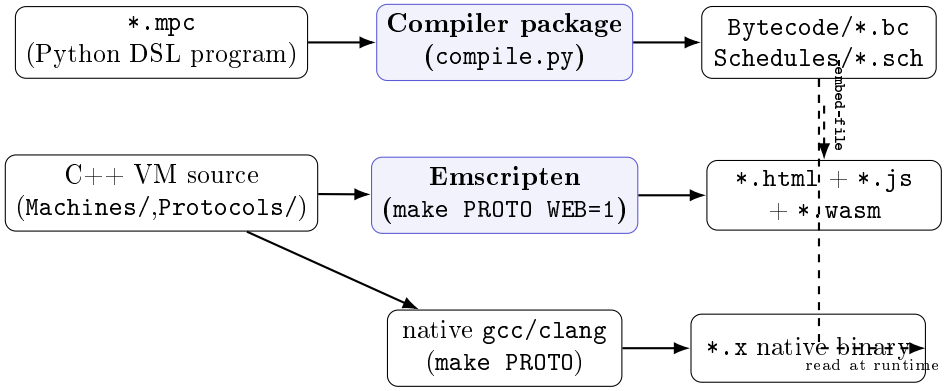


Figure 5.1: The two compilation stages: the Python **program compiler** (top path) turns an `.mpc` file into protocol-agnostic bytecode; the **C++ toolchain** (bottom path) turns the protocol virtual machine into either a native binary or a WASM/JS/HTML bundle. For the web build, the bytecode is embedded into the bundle at link time; for native builds, it is read from disk at run time.

5.1 Stage 1 — the program compiler: `.mpc` to bytecode

MPC programs are written in `Programs/Source/*.mpc`: ordinary Python that imports `Compiler.types` (for the secret type `sint` and friends) and `Compiler.library`. Running

```
./compile.py Programs/Source/surplus_matching_private.mpc
```

drives `Compiler/compilerLib.py`'s `Compiler` class, which:

1. `exec`s the `.mpc` file as Python, so every `sint(...)`, arithmetic operator, and `reveal_to(...)` call is actually a Python call into the **Compiler** package that records an instruction into an in-memory program graph — the program is never literally "run" with real numbers at compile time.
2. Applies the optimizer/allocator passes in `Compiler/allocator.py` and related modules (e.g. instruction scheduling, register allocation for the bytecode VM's abstract registers).
3. Emits two protocol-agnostic artifacts: `Programs/Bytecode/*.bc` (the actual instruction stream) and `Programs/Schedules/*.sch` (which bytecode files make up the program, and in what order/threading structure).

This stage knows nothing about ATLAS, WebAssembly, or browsers — the same bytecode is later interpreted by *whichever* protocol VM you link against. That is precisely why the same `surplus_matching_private` program can run under `atlas-party.x` with no source changes.

5.2 Stage 2 — the protocol VM: C++ to native or to WASM

The second compilation is ordinary C++ (via the `Makefile`), but it branches hard on the `WEB` variable:

```
ifeq ($(WEB), 1)
LIB = libwebSPDZ.a
SHAREDLIB = libwebSPDZ.so
LDFLAGS += -sWASMFS -sUSE_BOOST_HEADERS \
   --js-library deps/datachannel-wasm/wasm/js/webrtc.js \
   -sPROXY_TO_PTHREAD --post-js local/testing-post.js \
   -sUSE_PTHREADS -sINITIAL_MEMORY=500mb \
   -sEXPORTED_RUNTIME_METHODS=FS,callMain
endif
...
atlas-party.x: GC/AtlasSecret.o
...
%.x: Machines/%.o $(MINI_OT) $(SHAREDLIB)
    $(CXX) -o $@ $(CFLAGS) $^ $(LDLIBS) $(LDFLAGS) -sPTHREAD_POOL_SIZE=20 \
      $(SHAREDLIB) -o $(subst .x,,$@).html \
      --embed-file Player-Data --embed-file Programs \
      --shell-file webspdz-shell.html
endif
```

Listing 5.1: `Makefile`: the web-build branch (abridged)

Two consequences of this recipe matter for everything in Chapter 9:

- **The compiled program's input files are baked in.** `-embed-file Player-Data -embed-file Programs` packages whatever is on disk at *link time* straight into the WASM module. Nothing about the inputs can change afterwards without relinking — this is exactly the limitation the academic paper (§6.3) calls out, and that this fork later works around at runtime (§7.2).
- **The generated `atlas-party.html` is a build artifact, not hand-written.** It is produced fresh from `-shell-file webspdz-shell.html` on every `make -B atlas-party.x`. Editing the generated `.html` file directly is a dead end — the next rebuild overwrites it. Anyone touching the page must edit `webspdz-shell.html` instead.

Building the ATLAS party for the web, end to end, is therefore just:

```
./compile.py Programs/Source/surplus_matching_private.mpc
make atlas-party.x WEB=1 -j
```

which produces the triple `atlas-party.html`, `atlas-party.js`, `atlas-party.wasm` — the three files a browser actually downloads.

5.3 No local toolchain needed to just *run* the demo

Because the compiled `.html` / `.js` / `.wasm` triples (one per protocol) and the prebuilt `local/wasm64/lib/*.a` dependency archives are all committed to git — not gitignored — a fresh clone of this repository already contains a fully working, pre-built demo. No Emscripten installation, and no compilation step of any kind, is required merely to *serve* the existing build. This single

fact is the entire reason the Docker image in Chapter 8 can be a bare static-file server instead of a multi-gigabyte build image.

CHAPTER 6

Case Study: the Surplus Matching Application

6.1 The business problem

Four parties (framed in the demo's UI as four countries) each hold four private resource levels $x_{i,0}, \ldots, x_{i,3}$, $i \in \{0, 1, 2, 3\}$. Against a public target $T = 10$, a level below T is a *deficit* and a level above T is a *surplus*:

$$d_{i,k} = \max(T - x_{i,k},\, 0), \qquad s_{i,k} = \max(x_{i,k} - T,\, 0).$$

The goal is to pick, among the three possible ways to pair up four parties, the pairing that lets surpluses cover deficits as well as possible — and to tell each party *only* the identity of its own best partner, never any raw level, deficit, surplus, or the identity of anyone else's partner.

6.2 Why this needs to be "data-oblivious"

Inside an MPC program, every value of type `sint` is secret-shared: no party (and no line of the program) ever has the plaintext in hand to branch on. An ordinary `if x > y:` is impossible, because *which branch to take* is itself secret. MPC programs instead compute *both* outcomes and arithmetically select the right one with a secret 0/1 bit. Three idioms, all present in `surplus_matching_private.mpc`, make this practical:

```
def smax(a, b):
    c = a > b
    return b + c * (a - b)

def smin(a, b):
    c = a > b
    return a - c * (a - b)

def positive_part(x):
    return x * (x > 0)
```

Listing 6.1: Arithmetic max/min/positive-part on secret values

Here `a > b` is itself a secret comparison bit $c \in \{0, 1\}$ (ATLAS computes this through bit decomposition, not by actually branching), and the max/min formulas are just linear combinations of a, b and c — no control flow at all.

Pair metrics

For a pair (i, j), the program sums, across all four skills, how much of i's deficit j's surplus covers and vice versa:

```
def pair_metrics(def_i, sur_i, def_j, sur_j):
    benefit = sint(0)
    coverage = sint(0)
    worst = sint(0)
    for k in range(4):
        help_i_from_j = smin(def_i[k], sur_j[k])
        help_j_from_i = smin(def_j[k], sur_i[k])
        benefit += help_i_from_j + help_j_from_i
        coverage += (help_i_from_j > 0)
        coverage += (help_j_from_i > 0)
        rem_i = def_i[k] - help_i_from_j
        rem_j = def_j[k] - help_j_from_i
        worst = smax(worst, rem_i)
        worst = smax(worst, rem_j)
    useful = benefit > 0
    return benefit, coverage, worst, useful
```

Listing 6.2: `Programs/Source/surplus_matching_private.mpc`: per-pair metrics

This is computed for *all six* possible pairs among four parties — not just the three that end up in a matching — because which three pairs will matter is not known until the lexicographic comparison below resolves, and that resolution must itself happen without branching.

Secret lexicographic selection

With four parties there are exactly three perfect matchings: $M_0 = \{01, 23\}$, $M_1 = \{02, 13\}$, $M_2 = \{03, 12\}$. "Is A better than B?" is reduced to a single secret bit via mutually-exclusive priority terms (more matched parties, then higher benefit, then higher coverage, then lower worst remaining deficit):

```
def better(matched_a, benefit_a, coverage_a, worst_a,
           matched_b, benefit_b, coverage_b, worst_b):
    g1 = matched_a > matched_b; e1 = matched_a == matched_b
    g2 = benefit_a > benefit_b; e2 = benefit_a == benefit_b
    g3 = coverage_a > coverage_b; e3 = coverage_a == coverage_b
    g4 = worst_a < worst_b # lower worst deficit is better
    return g1 + e1*g2 + e1*e2*g3 + e1*e2*e3*g4
```

Listing 6.3: Secret lexicographic comparison, collapsed into one bit

The running "winning matching" and its metrics are then updated with a multiplexer pattern seen throughout MPC programs: $v \leftarrow c \cdot v' + (1 - c) \cdot v$, i.e. "keep the new value if the comparison bit c is 1, otherwise keep the old one" — again, no `if`.

Private, per-party output

Once a winning matching $\in \{0, 1, 2\}$ is selected (still secretly!), each party's partner is selected the same multiplexed way, and only then revealed — and only to the one party it belongs to:

```
def partner_if_useful(useful, partner_id):
    return -1 + useful * (partner_id + 1) # -1 if not useful, else partner_id

out0 = partner0.reveal_to(0)
```

```
out1 = partner1.reveal_to(1)
out2 = partner2.reveal_to(2)
out3 = partner3.reveal_to(3)
print_ln_to(0, 'YOUR PARTNER CODE = %s', out0)
print_ln_to(1, 'YOUR PARTNER CODE = %s', out1)
print_ln_to(2, 'YOUR PARTNER CODE = %s', out2)
print_ln_to(3, 'YOUR PARTNER CODE = %s', out3)
```

Listing 6.4: Private output: one `reveal_to` per party, nothing opened in common

`reveal_to(i)` opens a secret value *only* towards party i — the other three parties never see it, not even as ciphertext passing through their machine. No intermediate value (not even the winning matching's index) is ever opened publicly. This is the strongest security property the demo offers, and the most important sentence for a presentation: *every party learns a number; nobody, including the parties themselves, learns anybody else's inputs or anybody else's output.*

6.3 Worked example

The project's own draft paper, `paper/main.tex` ("*Private Partner Matching in the Browser: an Application of Multi-Party Computation with webSPDZ*"), documents a worked example that matches the inputs shipped in `Player-Data/Input-P{0,1,2,3}-0` exactly:

Table 6.1: Worked example from the paper, $T = 10$. Only the rightmost column is ever revealed, and each party sees only its own row of it.

Party	$x_{i,0}$	$x_{i,1}$	$x_{i,2}$	$x_{i,3}$	Output (partner)
0	4	13	6	10	3
1	15	3	12	7	2
2	8	17	5	11	1
3	12	8	14	4	0

Table 6.2: The three candidate matchings' secret metrics. M_2 wins on total benefit (20 vs 19) even though M_0 also matches all four parties.

	matched	benefit	coverage	worst remaining deficit
$M_0 = \{01, 23\}$	4	19	7	5
$M_1 = \{02, 13\}$	0	0	0	7
$M_2 = \{03, 12\}$	4	**20**	7	6

M_2 wins (highest total benefit among the two matchings that match all four parties), so party 0 is told "your partner is 3", party 1 is told "your partner is 2", and so on — exactly the output this fork's regression checks reproduced every time a tab ran this program (§9.5).

CHAPTER 7

System Architecture of the Running Demo

7.1 Three components, one trust boundary

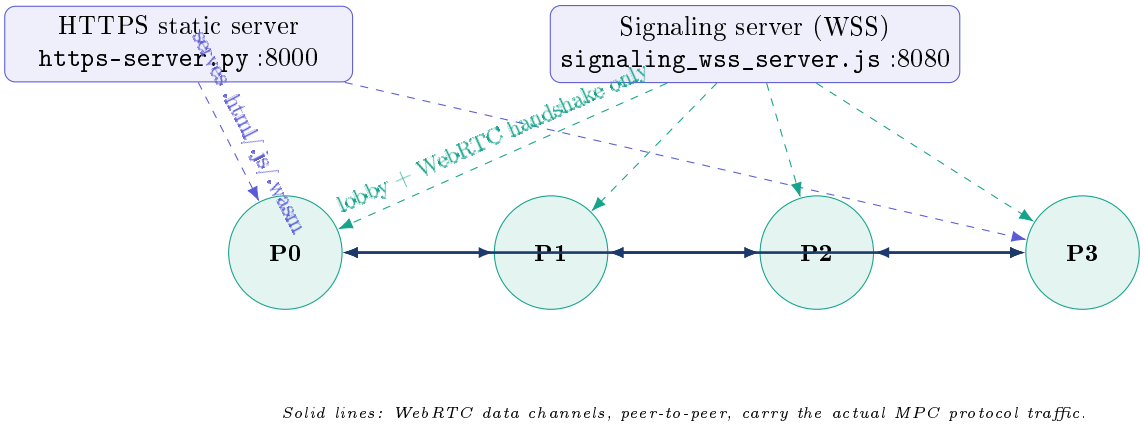


Figure 7.1: The three architectural components. The two servers never see protocol traffic; once the mesh of WebRTC channels is open, all MPC communication is direct, browser to browser.

1. **HTTPS static server** (`https-server.py`) serves the compiled page, JS and WASM, and the `COOP` / `COEP` cross-origin-isolation headers that browsers require before granting WASM the shared memory it needs for threads.
2. **Signaling server** (`signaling_wss_server.js`, Node.js over secure WebSockets) relays WebRTC `offer` / `answer` / `candidate` messages, and — in this fork's addition — a second, independent *lobby* protocol (§7.3). It is never on the path of a single protocol message once peers are connected.
3. **Four browser parties**, each running the same compiled `atlas-party.html` / `.js` / `.wasm` triple, with the MPC virtual machine executing in a dedicated application thread so the tab's UI stays responsive (§4).

7.2 The argv/input injection mechanism

This is the single most reused mechanism in the fork's own changes (Chapter 9), so it is worth detailing precisely how a party's program arguments and private input reach the compiled C++ `main()`.

1. **The page is told not to auto-start.** `webspdz-shell.html` sets `Module.noInitialRun = true` *before* the generated `.js` glue loads. Emscripten's runtime then initializes memory,

links the WASM module, and mounts the embedded filesystem — but does not call `main()` on its own.

2. `local/testing-post.js` **exposes a single JS entry point**, `window.webSPDZStartParty(args, inputFiles)`, injected via the shared `-post-js` flag into *every* `*-party.x` web target, not just ATLAS. `inputFiles` is a plain `{path: contents}` map, e.g. `{'Player-Data/Input-P0-0': '5\n5\n5\n'}`.
3. **It writes those files into the in-browser virtual filesystem** through `Module['FS']` (exported specifically for this purpose via `-sEXPORTED_RUNTIME_METHODS=FS,callMain`, §4.3), then calls `Module['callMain'](args)` — but *only* once `Module['onRuntimeInitialized']` has already fired, buffering the request in a `pending` variable if asked to start earlier.
4. `main()` **runs** exactly as it would natively, reading its argv and its input files from what looks, from the C++ side, like an ordinary local filesystem.

Backward compatibility is preserved in the same file: if the page URL still carries a legacy `?arguments=...` query string, `testing-post.js` calls `webSPDZStartParty` itself with no `inputFiles`, reproducing the original, purely-URL-driven behaviour used before this fork's lobby existed.

- **Editing the generated `atlas-party.html` does nothing** — it is regenerated by `-shell-file webspdz-shell.html` on every rebuild; edit the shell file instead (§5.2).
- **Calling `Module.callMain()` too early crashes** with `TypeError: Cannot read properties of undefined (reading '_a')` inside `__emscripten_stack_alloc` — the call races the WASM module's own asynchronous link step. Fix: only call it from inside `Module['onRuntimeInitialized']`.
- `FS.writeFile` **silently appends instead of truncating** on this exact Emscripten 4.0.6 + `-sWASMFS` + `-sMEMORY64` combination, when the target path already has content baked in by `-embed-file`. A genuine toolchain bug (`FS.stat()` throws `Cannot convert N to a BigInt` on the same path), not something '`{flags:'w'}`' or `FS.unlink()` fixes. Workaround: call `FS.truncate(path, 0)` (wrapped in `try/catch`) immediately before every `FS.writeFile()`.

7.3 The lobby protocol

`atlas-lobby.html` (a hand-written page, deliberately kept separate from the generated shell pipeline) turns the raw `webSPDZStartParty` hook into something a non-technical participant can use. `signaling_wss_server.js` carries a *second, independent* message protocol alongside its original WebRTC handshake (`login` / `offer` / `answer` / `candidate`): `lobby_join` → `lobby_assigned` / `lobby_error`, `lobby_ready`, `lobby_start`. The two conversations never collide because the WASM module's own networking (used once the MPC protocol itself starts) is keyed by `group`+`name`, independently of the lobby's room/seat state. The lobby protocol **never carries input values** — only room settings (size, threshold, program, chosen once by whoever's `lobby_join` created the room) and a per-seat `ready` boolean.

Seat assignment is a real race — by design

Which browser tab gets seat 0 vs. seat 3 is determined by which `lobby_join` message reaches the server first; four concurrently-opened tabs were observed landing in shuffled order (e.g. `[1,0,3,2]`). This is correct, intended behaviour, not a bug — `atlas-lobby.html` always reads back its *own* assigned seat from the server before writing its input file, so the party index used to start the WASM module is always right, even though the open order is not.

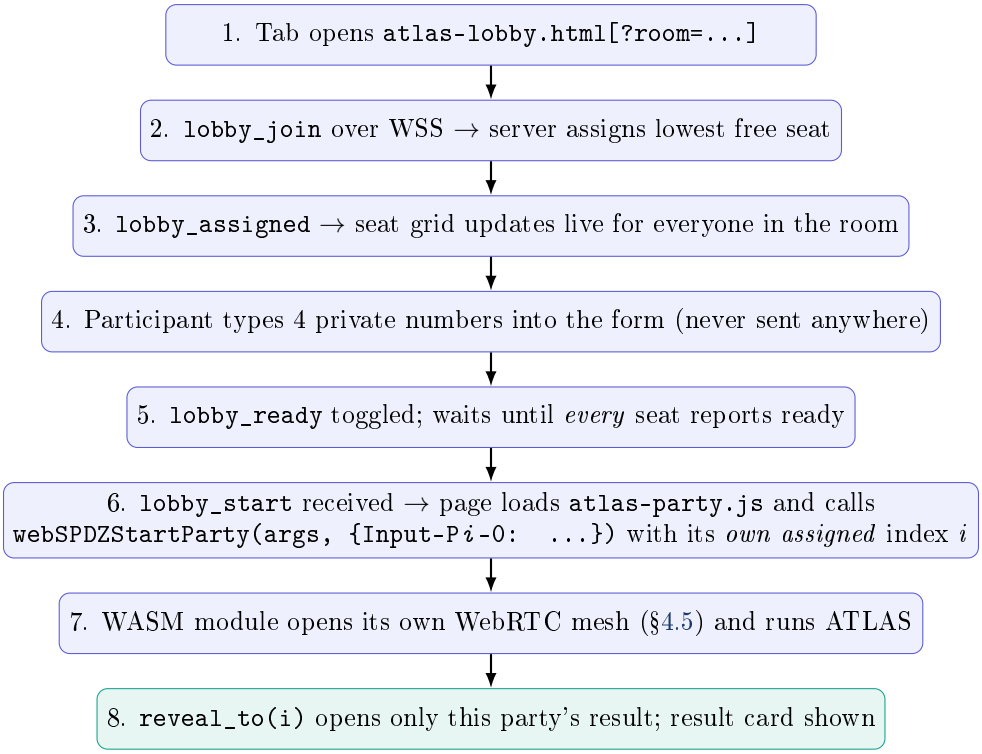


Figure 7.2: The end-to-end flow of a single lobby-driven run, from opening the shared link to seeing a private result.

CHAPTER 8

Docker Deployment

8.1 Why the image can be *serve-only*

As established in §5.3, every `*-party.html` / `.js` / `.wasm` triple this demo needs is already committed to the repository, built from whoever last ran `make -B atlas-party.x` on a dev machine. `Dockerfile.demo` is therefore deliberately *not* a build image: it installs only `python3` (for `https-server.py`) on a `node:20-bookworm-slim` base (for `signaling_wss_server.js` and its one dependency, `ws`), copies the already-built static files, and runs the two servers.

```
FROM node:20-bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends python3
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY *.html *.js *.wasm *.py ./
COPY certificates/ ./certificates/
EXPOSE 8000 8080
```

Listing 8.1: `Dockerfile.demo` (abridged)

This makes the image **architecture-independent by construction**: there is no C++/WASM compilation step inside the image that could behave differently on Apple Silicon vs. Intel vs. Linux, and the build itself is fast because nothing is actually compiled. A `.dockerignore` trims the repository (roughly 870 MB, mostly `Paper-Benchmarks/` and `.git`) down to just what serving needs, so the Docker build context stays small.

8.2 Two services, one image

```
services:
  web:
    build: { context: ., dockerfile: Dockerfile.demo }
    image: webspdz-demo:local
    command: ["python3", "https-server.py", "0.0.0.0", "8000"]
    ports: ["8000:8000"]
  signaling:
    image: webspdz-demo:local
    command: ["node", "signaling_wss_server.js", "0.0.0.0", "8080"]
    ports: ["8080:8080"]
    depends_on: [web]
```

Listing 8.2: `docker-compose.yml` (abridged)

Both services reuse the *same* built image (`webspdz-demo:local`) with a different `command:`, avoiding a second build entirely.

Both server scripts bind to whatever host string is passed as `argv[1]`. Bound literally to `localhost` / `127.0.0.1` *inside* the container, they become unreachable through Docker's `-p` port publishing, which connects to the container's external network interface, not its loopback. The fix is exactly what is shown above: pass `0.0.0.0` as the bind host in the Compose `command:` for both services. This only changes the *server-side* bind address — nothing in the browser-side JavaScript needs to change, since the browser itself runs on the host machine and correctly resolves `localhost:8000` / `:8080` to the published ports.

8.3 Running it

```
docker compose up --build
# then open, in four tabs:
# https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,0,
    surplus_matching_private,-OF,.
# https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,1,
    surplus_matching_private,-OF,.
# ... (party index 2, 3)
# or, for the lobby flow: https://localhost:8000/atlas-lobby.html
```

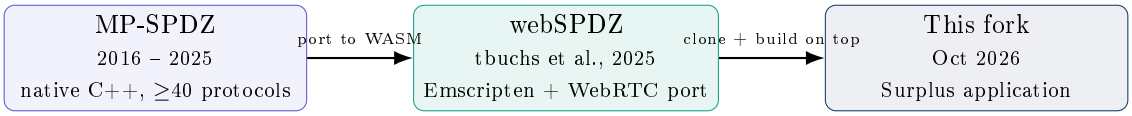
Chrome/Firefox will warn about the self-signed certificate on first visit to both `:8000` and `:8080` (the signaling server also needs the certificate trusted, since it speaks `wss://`) — accept the exception on each before opening the party tabs.

CHAPTER 9

Project History: From the First Clone to the Demo

9.1 Lineage

The repository's git history is linear and unusually long-lived: it begins in **2016** as the original SPDZ/MP-SPDZ project, accumulates contributions from dozens of academic contributors over nearly a decade, becomes **tbuchs/webSPDZ** (the Emscripten/WebRTC port described in Chapter 4, published as ePrint 2025/487 [1]), and is then cloned into *this* repository, where the work described below was done on top of it.



This fork's own history is exactly **eight commits**, authored between 2026-10-06 and 2026-10-23, all on top of upstream commit `3072e5a7`. Table 9.1 lists them in order; the rest of this chapter walks through each in the depth needed for a presentation.

Table 9.1: All eight commits authored on top of upstream webSPDZ, oldest first.

Hash	Date		Commit message
`ba257011`	2026-10-06	§9.2	Functional and styled version
`56734a97`	2026-10-22	§9.3	Functional
`087c399a`	2026-10-22	§9.4	Dockerization and guide
`3bf9c837`	2026-10-22	§9.5	Without arguments in the url, but with entrance in the console
`f99b28d6`	2026-10-22	§9.6	Now with custom inputs in the web
`b160b884`	2026-10-22	§9.7	Guide how to use it without arguments at the url
`d593fadd`	2026-10-23	§9.8	Frontend for how it works
`cfabfe67`	2026-10-23	§9.9	How it works section

9.2 Commit 1 — `ba257011`: the application is born

This is the commit that turns generic webSPDZ into *Surplus*. It adds, in one step: the entire `surplus_matching_private.mpc` program (230 lines, Chapter 6), the four parties' `Player-Data/Input-P{0..3}-0` files with the worked-example values, a pre-built `atlas-party.wasm` linking that program in, and a substantial rewrite (484 lines changed) of `webspdz-shell.html` to replace Emscripten's bare default page with a styled front-end. At this point, the only way to run the

demo is still the original mechanism: a hand-built `?arguments=...` URL per tab, with every party's input permanently baked into the `.wasm` at compile time.

9.3 Commit 2 — `56734a97`: housekeeping

A small commit: a `.gitignore` tweak and committing a user-supplied reference PDF (`Guia_desde_cero_Surplus_ATLAS.pdf`). Not functionally significant, but marks a 16-day gap in the timeline between the first working version and the documentation/Docker/lobby work that follows.

9.4 Commit 3 — `087c399a`: Dockerization

Adds `Dockerfile.demo`, `docker-compose.yml`, `.dockerignore`, and a Docker guide PDF — exactly the serve-only image described in Chapter 8. The key discovery driving this commit's design (that the repository already ships a fully pre-built demo, §5.3) is what makes the image so simple; the key bug fixed in it is the `0.0.0.0` vs. `localhost` bind-address gotcha.

9.5 Commit 4 — `3bf9c837`: the input-at-runtime prototype

This is the commit that resolves the paper's own stated #1 limitation ("*Inputs at build time*", §6.3). It builds the deferred-`main()` and `FS`-injection mechanism documented in full in §7.2: `Module.noInitialRun` in `webspdz-shell.html`, the new `window.webSPDZStartParty` entry point in `local/testing-post.js`, and the `Makefile` change exporting `FS,callMain` to page JavaScript. The scope was deliberately bounded up front: still a single-machine, four-tab demo (not four separate computers), and input values must never leave the browser they are typed into — only readiness/coordination signals may reach a server. This commit is also where all three debugging war-stories in the warning box of §7.2 were found and fixed. It was verified with ad hoc Playwright scripts driving the system's installed Chrome, checking both that legacy `?arguments=...` URLs still reproduce the paper's worked example, and that four tabs started with *no* URL parameters at all — driven purely by `webSPDZStartParty` with hand-chosen, hand-verifiable input profiles — produce the independently-computed expected result. None of these test scripts were committed to the repository.

9.6 Commit 5 — `f99b28d6`: the lobby

Turns the raw prototype from commit 4 into something a non-technical participant can actually drive: the `lobby_*` message protocol added to `signaling_wss_server.js`, and the first version of `atlas-lobby.html` (§7.3) — one shared link, automatic seat assignment, a form for each participant's own four numbers, and a "ready" toggle that starts the computation once every seat reports ready.

9.7 Commit 6 — `b160b884`: the beginner's guide

A documentation-only commit: a plain-language, click-by-click PDF guide assuming zero technical background, covering Docker installation through reading a private result, using the new lobby so that no console or URL-editing is ever required.

9.8 Commit 7 — `d593fadd`: the lobby redesign

A near-total visual rewrite of `atlas-lobby.html` (897 lines touched out of 769 total) plus small protocol tweaks in `signaling_wss_server.js`. The generic "lobby" becomes **Surplus — Skills**

without borders: parties are re-framed as countries (with flag emoji and a country picker), the layout gains a persistent sidebar ("Mi país" / "Mapa global" / "Red de alianzas" / "Cómo funciona", the latter three marked *próximamente* — coming soon) and a skills form styled as labelled sliders rather than bare number inputs. Functionally this commit keeps the exact same WebSocket lobby protocol and the exact same `webSPDZStartParty` call underneath; everything that changed is presentation, which is precisely why it was safe to do without touching the signaling protocol's message shapes beyond minor additions.

9.9 Commit 8 — `cfabfe67`: the explainer page

Adds `atlas-how-it-works.html` (291 new lines), a new static page reachable from the sidebar's "Cómo funciona" link added in the previous commit. It explains the demo in four non-technical steps — entering capabilities privately, computing with ATLAS peer-to-peer, receiving a private recommendation, and an (explicitly "coming soon") opt-in public alliance map — plus a small JS-drawn mesh diagram of four nodes, fully connected, deliberately chosen to mirror the *real* WebRTC topology (every pair of parties connects directly; there is no central node) rather than a misleading hub-and-spoke picture. This page is pure front-end: it adds no new server messages and calls no WASM code at all.

9.10 What this fork changed vs. what it inherited, in one table

Table 9.2: Attribution summary, aligned with Chapters 3–8.

Layer	Origin
ATLAS protocol math (`Protocols/Atlas*`)	Unmodified, inherited from MP-SPDZ
Python compiler, bytecode VM	Unmodified, inherited from MP-SPDZ
Emscripten/WebRTC port, threading model	Inherited from tbuchs/webSPDZ (ePrint 2025/487)
`surplus_matching_private.mpc` application	**This fork**, commit `ba257011`
Deferred-`main()` / `FS` runtime input injection	**This fork**, commit `3bf9c837`
Lobby protocol & UI, country-themed redesign	**This fork**, commits `f99b28d6`, `d593fadd`
"How it works" explainer page	**This fork**, commit `cfabfe67`
Docker packaging	**This fork**, commit `087c399a`

CHAPTER 10

Security Discussion and Limitations

10.1 What is actually revealed

Against a semi-honest adversary corrupting at most one of the four parties (the $t = 1$, $n = 4$ ATLAS threshold), a party learns only its own partner identity — but it does learn what that output *implies*: being paired with j reveals that pair had positive benefit, and that the overall selected matching ranked best among the three candidates. No raw resource level, deficit, surplus, or any other party's partner is ever opened to anyone.

10.2 Known limitations

- **Fixed size.** The three perfect matchings are hand-enumerated for $n = 4$ only. For larger n the number of perfect matchings grows as $(n - 1)!!$, so a genuinely different (and more complex, likely sorting-network-based) algorithm would be needed to scale the application.
- **Semi-honest only.** ATLAS protects against nosy parties, not against parties that actively deviate from the protocol. An actively-secure variant (e.g. Rep4-Ring, already in Table 2.1) would be needed for a threat model that includes malicious participants.
- **Trust in the two servers, as deployed.** While neither server sees protocol traffic (§7), a malicious signaling server could in principle try to man-in-the-middle the WebRTC handshake itself; this is not yet separately evaluated.
- **Inputs-at-build-time, for the *legacy* URL path only.** The original limitation the paper describes is resolved for the lobby flow (§7.2), but the old `?arguments=...` URL mechanism is kept for backward compatibility and testing, and still relies on whatever was baked into the build at compile time.

CHAPTER 11

Study Guide: Key Concepts, Glossary and Presentation Q&A

11.1 One-paragraph answers, ready to say out loud

"What is MPC, in one sentence?" Several parties jointly compute a function of their private inputs so that nobody learns anything about anyone else's input beyond what the output necessarily reveals — and here, no server learns anything at all.

"Why ATLAS specifically?" It is an honest-majority, Shamir-based protocol whose multiplication step is cheap: one round-robined "king" per multiplication reconstructs a one-time-masked value and re-shares it, instead of running a full interactive resharing sub-protocol every time. That fits a four-party, one-corruption demo well.

"How does a browser, with no raw sockets, do networking?" WebRTC data channels, peer-to-peer, bootstrapped by a signaling server that only relays handshake messages and never sees protocol traffic.

"Why WebAssembly instead of rewriting the protocol in JavaScript?" MP-SPDZ is $\geq$40 protocols' worth of tested, performance-tuned C++. Emscripten lets that exact code run in-browser, at near-native speed, instead of a risky, from-scratch reimplementation.

"Is anything baked into the compiled file?" Yes, by default — Emscripten's `-embed-file` packages `Player-Data` and `Programs` into the WASM module at *link* time. This fork's lobby mechanism writes a fresh value over that baked-in file at *run* time, in the browser's own virtual filesystem, before calling `main()`.

"What exactly does the signaling server see?" WebRTC offers, answers, and ICE candidates — opaque connection-setup data, not protocol messages — plus, in this fork, lobby room/seat/ready state. Never an input value.

"Why is the Docker image so small/fast to build?" Because the compiled WASM/JS/HTML artifacts and their native library dependencies are already committed to git; the image just serves static files and runs two small Node/Python servers, with no C++ toolchain inside it at all.

"What, concretely, did *this* fork add on top of webSPDZ?" One application (the surplus-matching program), a way to type private input live instead of baking it in, a lobby UI with automatic seat assignment, a Docker packaging of the whole thing, and an explainer page — summarized in the attribution table at the end of Chapter 9.

11.2 Glossary

MPC Secure multi-party computation: jointly computing a function of private inputs without revealing them.

Secret sharing Splitting a value into shares such that a threshold number of shares reconstructs it, and fewer reveal nothing.

Shamir secret sharing Secret sharing via polynomial evaluation; degree t tolerates t passive corruptions.

Honest majority A security model where at most half the parties can be corrupted.

Passive/semi-honest A corrupted party follows the protocol but tries to learn extra information from what it legitimately sees.

Active/malicious A corrupted party may deviate arbitrarily from the protocol.

ATLAS The honest-majority, Shamir-based MPC protocol used by this demo; cheap multiplication via precomputed double sharings and a round-robin "king".

`reveal_to(i)` An MP-SPDZ primitive that opens a secret value only towards party i.

Emscripten The LLVM-based toolchain that compiles C/C++ to WebAssembly (plus supporting JavaScript glue).

WASM / WebAssembly A portable, sandboxed binary instruction format that runs in the browser at near-native speed.

WebRTC A browser API for real-time, peer-to-peer audio/video/data connections; this project uses only its data channels.

Signaling server A server used solely to exchange the connection metadata two WebRTC peers need before they can talk directly.

Bytecode / schedule The protocol-agnostic output of MP-SPDZ's Python compiler: the instruction stream (`.bc`) and its execution plan (`.sch`).

`-embed-file` An Emscripten linker flag that packages files from disk into the compiled WASM module at link time.

`-shell-file` An Emscripten linker flag that generates the output HTML page from a custom template instead of Emscripten's default.

APPENDIX A

Command Reference

```
# Stage 1: Python DSL -> bytecode/schedule (protocol-agnostic)
./compile.py Programs/Source/surplus_matching_private.mpc

# Stage 2a: native build (any protocol, e.g. Shamir)
make shamir -j

# Stage 2b: web build of ATLAS specifically
make atlas-party.x WEB=1 -j
#   -> atlas-party.html, atlas-party.js, atlas-party.wasm

# Serve it locally (native dev servers, no Docker)
./run-servers.sh
#   equivalent to:
#   python3 https-server.py 0.0.0.0 8000
#   node signaling_wss_server.js 0.0.0.0 8080

# Or via Docker:
docker compose up --build
```

Listing A.1: Compile the MPC program and build a native or web party

```
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,0,
    surplus_matching_private,-OF,.
# -N,4  : four parties    -T,1 : threshold 1
# -w,0  : running in-browser; 0 = WebRTC transport (1 = WebSockets)
# next arg: this tab's party index (0..3)
# then : program name (surplus_matching_private)
# -OF,. : output-file directory (web build convention)
```

Listing A.2: Legacy URL-argument invocation (one tab per party)

```
https://localhost:8000/atlas-lobby.html?room=atlas-demo
# open 4 tabs on the same room; each: joins, gets a seat, fills in 4
# private numbers, clicks "ready"; computation starts once all 4 are ready.
```

Listing A.3: Lobby invocation (no URL editing, any number of tabs)

APPENDIX B

References

Bibliography

[1] T. Buchsteiner, K. W. Koch, D. Rotaru, and C. Rechberger. webSPDZ: Versatile MPC on the Web. Cryptology ePrint Archive, Paper 2025/487, 2025. https://eprint.iacr.org/2025/487

[2] M. Keller. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *ACM CCS*, 2020.

[3] V. Goyal, H. Li, R. Ostrovsky, A. Polychroniadou, and Y. Song. ATLAS: Efficient and Scalable MPC in the Honest Majority Setting. In *CRYPTO*, 2021.

[4] Y. Lindell. Secure Multiparty Computation. *Communications of the ACM*, 64(1):86–96, 2021.